Reviewer Pack — Minimal Local Induction Ring Rank and Communication Complexi…

Entry 71a6dbf6983b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 23:11:27 UTC

Minimal Local Induction Ring Rank and Communication Complexity Rank Variance

Entry ID: `71a6dbf6983b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 23:11:27 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Local Induction Rings) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every circuit C with n inputs, the variance of the local induction ring rank over all possible evaluations is O(n log n), where the evaluation is defined as the matrix representation of the circuit.

Rationale (proposer's reasoning):

> Local induction rings provide a framework to study algebraic properties of circuits. The conjecture suggests that the complexity of communication complexity can be understood through the algebraic structure provided by these rings, potentially revealing deeper connections between algebra and complexity theory.

Taxonomy category: `local_induction_ring_rank` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `19045bb1b3da86ad`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every circuit C with n inputs, the variance of the local induction ring rank over all possible evaluations is considered O(n log n) if the computed variance falls within a confidence interval [0, 10 * (n log n)] for at least 25 out of 30 random seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `algebra local induction rings AND communication complexity matrix rank` - `local induction ring rank variance IN algebra AND communication complexity` - `matrix representation circuits AND local induction rings rank O(n log n)`

Top relevant hits considered: - [http://arxiv.org/abs/1401.4438v1] Integral closure of rings of integer-valued polynomials on algebras - [http://arxiv.org/abs/1501.07530v3] Some algebras similar to the 2x2 Jordanian matrix algebras - [http://arxiv.org/abs/1305.1444v6] The Green rings of the 2-rank Taft algebra and its two relatives twisted - [http://arxiv.org/abs/2411.11095v3] Invariant theory and coefficient algebras of Lie algebras - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/2111.06855v3] Response of a CMS HGCAL silicon-pad electromagnetic calorimeter prototype to 20-300 GeV positrons - [http://arxiv.org/abs/1304.7516v1] Quantum Circuits for GCD Computation with $O(n \log n)$ Depth and O(n) Ancillae

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_circuit(n):
    circuit = []
    for _ in range(2**n):
        gate = random.choice(['AND', 'OR'])
        inputs = random.sample(range(n), 2)
        circuit.append((gate, inputs))
    return circuit

def evaluate_circuit(circuit, inputs):
    stack = inputs[:]
    for gate, inputs in reversed(circuit):
        if len(stack) < 2:
            raise ValueError("Invalid circuit: not enough values on the stack")
        a = stack.pop()
        b = stack.pop()
        if gate == 'AND':
            result = a and b
        elif gate == 'OR':
            result = a or b
```

```
            else:
                raise ValueError(f"Invalid gate: {gate}")
            stack.append(result)
        if len(stack) != 1:
            raise ValueError("Invalid circuit: too many values on the stack")
        return stack[0]

    def local_induction_ring_rank(circuit):
        n = len(circuit)
        rank = 0
        for i in range(n):
            inputs = [0] * n
            inputs[i] = 1
            if evaluate_circuit(circuit, inputs):
                rank += 1
        return rank

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        n_values = [5, 10, 15, 20, 30, 40]
        total_variance = 0
        instances_tested = 0
        n_max = 0
        conjecture_holds = True
        counterexample = ""

        for n in n_values:
            circuits = [generate_random_circuit(n) for _ in range(5)]
            ranks = [local_induction_ring_rank(circuit) for circuit in circuits]
            variance = sum((x - sum(ranks) / len(ranks)) ** 2 for x in ranks) / len(ranks)
            total_variance += variance
            instances_tested += len(ranks)
            n_max = max(n_max, n)

            if variance > 10 * (n * math.log(n)):
                conjecture_holds = False
                counterexample = f"Variance {variance} exceeds 10 * {n} * log({n})"

        mean_variance = total_variance / len(n_values)
        support_fraction = sum(1 for n in n_values if variance <= 10 * (n * math.log(n))) / len(n_valu

        return {
            "metric_name": "Variance of Local Induction Ring Rank",
            "metric_value": mean_variance,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

    if __name__ == "__main__":
        import sys
        seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

        results = []
        for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_variance = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if not r["counterexample"]) / len(results)

    if all(not r["counterexample"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_variance} std=0.0 support_fraction={support_fraction}
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_variance} std=0.0 support_fraction={support_fraction}
    else:
        first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b0b4ef6.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b0b4ef6.py", line 65, in run_trial
    ranks = [local_induction_ring_rank(circuit) for circuit in circuits]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b0b4ef6.py", line 50, in local_inducti
    if evaluate_circuit(circuit, inputs):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b0b4ef6.py", line 30, in evaluate_circ
    raise ValueError("Invalid circuit: not enough values on the stack")
ValueError: Invalid circuit: not enough values on the stack
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution due to an invalid circuit error, which prevented the computation of the variance and its comparison against the conjectured bound. | next: Review the implementation of evaluate_circuit to ensure it handles all possible circuit inputs correctly and consider running the test again with a valid circuit.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13780
2	preregistration	ollama_remote	glm4:latest	0	0	15009
3	novelty	ollama_remote	glm4:latest	0	0	8395
4	novelty	ollama_remote	glm4:latest	0	0	10724
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14519
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9438
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10816
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10401
9	judge	ollama_remote	glm4:latest	0	0	28122

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121204 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/71a6dbf6983b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/71a6dbf6983b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/71a6dbf6983b.tar.gz` (if generated)